

00 / ĐỊNH HƯỚNG

Hiểu bài giảng, từng câu một.

Lộ trình học tập và triển khai mini project cá nhân • Tên làm việc: LectureBridge

Xây dựng hệ thống nhận âm thanh tiếng Anh trong giảng đường Mỹ, hiện transcript tiếng Anh và bản dịch tiếng Việt trên iPhone/iPad. Laptop ở ký túc xá thực hiện toàn bộ suy luận AI; thiết bị di động thu mic và đọc phụ đề.

Thiết bị mục tiêu

Lenovo IdeaPad Gaming 3 • Intel i5-12500H • RTX 3050 4 GB VRAM • RAM 16 GB • Ubuntu 24.04 dual boot.

Kết quả cần đạt

Một bản thử nghiệm có thể chạy xuyên suốt buổi học 60-90 phút, hiển thị rõ bản nháp và câu đã chốt, tự báo khi mất kết nối, có số liệu về độ trễ và chất lượng. Repo GitHub phải giải thích được cách chạy lại và các giới hạn thực tế.

Kỳ vọng thực tế

Đây là công cụ hỗ trợ hiểu bài, với độ trễ tính bằng giây. Chưa có benchmark trên máy của bạn: mọi ngưỡng trong tài liệu là **mục tiêu nghiệm thu đề xuất**, không phải hiệu năng đã đo. Micro xa giảng viên và Wi-Fi yếu có thể là điểm nghẽn lớn hơn model.

Ưu tiên phần mềm không cần API trả phí. Vẫn có chi phí điện, Internet, dung lượng tải model và bảo trì. “Local-first” ở đây nghĩa là suy luận trên laptop của bạn; khi laptop ở dorm, âm thanh vẫn đi qua mạng riêng.

Cách đọc tài liệu

Trang 2-5: phạm vi, lựa chọn công nghệ và kiến trúc. Trang 6-8: học và làm theo từng giai đoạn. Trang 9-10: đánh giá và kiểm thử. Trang 11-13: vận hành, quyền riêng tư và GitHub. Trang 14: nguồn kỹ thuật.

01 / PHẠM VI

Một người dùng, một luồng âm thanh

MVP: phần nhỏ nhất nhưng có ích

Có trong bản đầu	Để sau khi MVP ổn định
Mic từ một thiết bị; EN và VI cùng màn hình.	Nhận diện nhiều người nói, dịch giọng nói ra loa.
Start/Stop, trạng thái kết nối, bản nháp/câu chốt.	Tóm tắt bằng LLM, hỏi đáp slide, RAG.
iPhone thu mic; iPad có thể chỉ xem cùng phiên.	Nhiều lớp hoặc nhiều người dùng đồng thời.
Chạy riêng qua Tailscale; lưu transcript theo lựa chọn.	Ứng dụng iOS native, App Store, thương mại hóa.

Ba nguyên tắc thiết kế

- 1. Có tiếng Anh làm điểm tựa.** Luôn cho xem nguồn EN cạnh VI; đánh dấu đoạn thiếu âm thanh. Bản dịch mượt không đồng nghĩa đúng nghĩa, nhất là phủ định, đơn vị và thuật ngữ.
- 2. Ổn định trước tối ưu.** Đầu tiên dịch cụm EN đã ổn định. Sau đó mới thử NLLW để dịch tăng dần; không dịch lại toàn bộ lịch sử mỗi khi có một từ mới.
- 3. Đo trước khi nâng model.** Cùng dữ liệu, cùng cấu hình mic, so sánh tốc độ và lỗi. Không mặc định model lớn hơn sẽ cải thiện trải nghiệm trên VRAM 4 GB.

Quyết định trước phiên thử tại lớp

Xác nhận quy định của môn học về thu và xử lý âm thanh với giảng viên. Chỉ dùng dữ liệu tự tạo hoặc được phép trong bộ thử nghiệm. Xác minh Wi-Fi trường cho kết nối đến laptop; có phương án ngồi gần nguồn âm và phương án transcript EN khi dịch bị chậm.

Định nghĩa hoàn thành: một phiên 90 phút qua mạng khác dorm, không crash, không tích lũy độ trễ kéo dài, không mất đoạn mà không thông báo; kèm báo cáo chất lượng và hướng dẫn tái lập.

02 / CÔNG NGHỆ

Chọn cấu hình vừa sức máy

Thành phần	Khởi đầu đề xuất	Lý do / điểm cần xác minh
ASR	WhisperLiveKit; thử base.en, rồi small.en	WLK lo vòng đời streaming. Xác nhận backend và model được bản đã ghim hỗ trợ. [1]
Suy luận ASR	Faster-Whisper / CTranslate2 trên GPU	Thử int8_float16, batch 1 nếu backend hỗ trợ. Đây là engine, không tự tạo chính sách streaming. [2]
Dịch EN → VI	NLLB-200 distilled 600M, CPU INT8	Dành VRAM cho ASR; kiểm chứng chuyển đổi CTranslate2 và tokenizer. [3]
Dịch tăng dần	NLLW sau baseline	Lớp dịch đồng thời dựa trên NLLB; không phải một model độc lập thay NLLB. [4]
Phát hiện lời nói	VAD có sẵn; Silero khi cần	Chỉ một nơi chịu trách nhiệm VAD để tránh cắt hai lần. [5]
Ứng dụng	FastAPI + WebSocket; web/PWA	Dùng giao diện WLK để thử trước, rồi thêm UI và hợp đồng sự kiện riêng. [6]
Kết nối	Tailscale + Serve HTTPS	Chỉ phục vụ trong tailnet; iPhone/iPad cài Tailscale. [8]

Quy tắc nâng / hạ cấu hình

Nếu ASR base.en đáp ứng tốc độ nhưng sai thuật ngữ, thử small.en trên cùng bộ test. Nếu GPU hết bộ nhớ, hạ model và giảm buffer trước; không nạp thêm model dịch lên GPU. Nếu CPU dịch chậm, tăng độ dài cụm vừa phải, giới hạn luồng CPU và đo lại. Giữ chế độ EN-only khi backlog vượt ngưỡng.

Chưa chọn medium/large, NLLB 1.3B, diarization hoặc LLM cho MVP. NLLB sử dụng giấy phép CC-BY-NC-4.0; repo mã nguồn của bạn không thay thế giấy phép trọng số. [3]

03 / KIẾN TRÚC

Âm thanh đi đâu, chữ xuất hiện ở đâu?

iPhone: mic và nút Start

getUserMedia → AudioWorklet → audio frame + sequence

Mạng riêng: HTTPS / WSS

Tailscale trên thiết bị → Tailscale Serve tại laptop

Laptop: audio → tiếng Anh

Chuẩn hóa audio → VAD → streaming ASR → EN draft / committed

Laptop: tiếng Anh → tiếng Việt

Cụm ổn định → NLLB; giai đoạn sau so sánh NLLW

iPhone / iPad: giao diện song ngữ

WebSocket events → thay bản nháp, giữ câu đã chốt

Ranh giới triển khai

Bắt đầu bằng một dịch vụ ứng dụng, một worker suy luận và các module tách biệt. Vòng lặp WebSocket không chạy tác vụ model nặng trực tiếp: dùng worker/queue có giới hạn để tránh chặn heartbeat và mọi client. Nếu dùng WLL có sẵn, adapter ánh xạ giao thức của phiên bản đã ghim sang hợp đồng của dự án.

Chỉ một thiết bị giữ quyền gửi mic mỗi phiên. iPad đăng ký nhận kết quả từ cùng session, không chạy một pipeline ASR khác. Trong MVP, audio nằm trong buffer RAM hữu hạn; transcript chỉ ghi xuống đĩa khi người dùng bật lưu.

04 / STREAMING

Thiết kế để chậm mà không bị dón

Hợp đồng dữ liệu tối thiểu

Frame âm thanh: **session_id**, **seq**, **sample_rate**, **channels**, **sample_count** và payload nhị phân. Kết quả: **segment_id**, **revision**, **audio_start_ms**, **audio_end_ms**, **kind**, **text**; kind gồm **en_partial**, **en_final**, **vi_partial**, **vi_final**, **gap** và **status**. UI cập nhật theo ID/revision để không nhận đôi câu.

Một dòng chữ có hai trạng thái

Partial có thể đổi và hiển thị nhậ; **final** là phần đã commit theo chính sách của pipeline. Final không có nghĩa chắc chắn đúng. Chờ cụm đủ nghĩa để dịch; nếu sau này sửa câu chốt, gửi sự kiện sửa rõ ràng thay vì âm thầm ghi đè.

Điểm khởi đầu để thí nghiệm, không phải mặc định thư viện

Tham số	Giá trị thử đầu	Điều cần quan sát
Audio transport	100-250 ms/frame	Đây là nhíp gửi audio, không phải nhíp decode ASR.
Audio chuẩn	Mono PCM16, 16 kHz	Resample thật từ sample rate thực; không chỉ đổi nhãn.
VAD endpoint	Im lặng 400-700 ms	Đừng làm mất âm đầu/cuối; thêm pre-roll 200 ms.
Dịch baseline	Cụm ổn định; đợi tối đa 2-4 s	Cắt ở ranh giới có nghĩa; đo lỗi với câu chưa kết thúc.
Giới hạn queue	Tối đa 5 s audio đang chờ	Báo lag; chủ động bỏ đoạn có đánh dấu nếu quá tải.

Với PCM16 mono 16 kHz, payload là 32 KB/s, khoảng 115 MB/giờ trước overhead. AudioWorklet là đường thử ưu tiên; nếu dùng MediaRecorder, kiểm tra MIME thực trên Safari và giải mã stream đúng container. Một chunk nén không nhất thiết là file độc lập. [7]

Khi reconnect: tạo epoch mới, gửi seq/ACK cuối, không phát lại bản chốt trùng. Chỉ replay buffer ngăn được quản lý; phần vượt giới hạn trở thành gap. Flush phần còn lại khi Stop, đặt timeout và báo nếu chưa xử lý hết.

05 / HỌC TRƯỚC KHI LÀM

Giai đoạn 0-1: nền tảng và baseline

Ước lượng cá nhân: 2 tuần, 12-18 giờ. Nếu mới học Python, dành thêm 2-4 tuần cho nền tảng.

Giai đoạn 0 / Chuẩn bị môi trường

Học trước: Python hàm/module, type hints, exception, generator; terminal Linux, virtualenv, Git commit/branch; phân biệt RAM/VRAM, driver/CUDA/cuDNN và trọng số/runtime. Không cần học huấn luyện neural network trước.

Bài tập: viết chương trình đọc WAV và in thời lượng/sample rate; tạo môi trường riêng rồi tái tạo lại từ lockfile; tạo một nhánh sửa README và tự review diff. Lưu ghi chú lỗi cài đặt đã gặp.

Làm: kiểm tra NVIDIA driver với nvidia-smi; chạy thử model nhỏ trên GPU; ghi phiên bản driver, Python, CTranslate2 và thư viện CUDA. Theo tài liệu Faster-Whisper hiện tại, runtime GPU dùng CUDA 12 và cuDNN 9; đối chiếu lại với phiên bản ghim. [2]

Cổng M0: sau reboot Ubuntu vẫn nhận GPU; chạy được script 1 phút audio và tái tạo môi trường từ hướng dẫn. Không sửa phân vùng dual boot để làm dự án này.

Giai đoạn 1 / File audio → EN → VI

Học trước: ASR là nhận dạng lời nói; MT là dịch văn bản; VAD là tìm đoạn có lời nói. Học tokenization, lượng tử hóa và lý do lỗi ASR truyền sang bản dịch.

Bài tập: tự thu 2-3 phút tiếng Anh; chép tay 20 câu và dịch tham chiếu. Chạy base.en và small.en; dịch EN chuẩn rồi dịch EN do ASR tạo, so sánh riêng hai nguồn lỗi.

Làm: baseline Faster-Whisper offline + NLLB 600M; đặt nguồn eng_Latn, đích vie_Latn. Ghi thời gian load riêng thời gian inference. Lưu cấu hình và báo cáo, chưa xây PWA.

Cổng M1: xử lý xong 10 phút audio, có EN/VI theo đoạn và bảng lỗi; model không OOM. Chọn cấu hình dựa trên máy thật. Nếu dịch sai phủ định/số, ghi thành regression case trước khi nối live.

06 / HỌC VÀ TRIỂN KHAI

Giai đoạn 2-3: streaming rồi dịch

Ước lượng: 2-3 tuần, 16-24 giờ. Chỉ đi tiếp khi công nghiệm thu trước đã đạt.

Giai đoạn 2 / Stream tiếng Anh ổn định

Học trước: sample rate, PCM, frame/chunk, resampling, VAD; async/await, producer-consumer, bounded queue và WebSocket. Hiểu cửa sổ chống lẩn và cơ chế chốt phần chữ ổn định.

Bài tập: phát lại WAV theo nhịp thời gian thực, chia frame 200 ms; thêm delay ngẫu nhiên và in queue age. Viết WebSocket echo gửi sequence và phát hiện frame thiếu.

Làm: chạy demo WhisperLiveKit trên laptop. Đọc cấu hình backend của bản đã ghim; không giả định mọi backend đều dùng Faster-Whisper hoặc hỗ trợ cùng compute_type. Gắn adapter trả EN draft/final, timestamp và trạng thái lag. [1]

Cổng M2: replay 20 phút không lặp câu final, queue không tăng liên tục; silence không sinh đoạn chữ kéo dài. Có biểu đồ latency và log thiếu frame.

Giai đoạn 3 / Thêm tiếng Việt có kiểm soát

Học trước: segmentation, stable prefix, lỗi thiếu ngữ cảnh, đánh đổi latency/quality. NLLW cung cấp cơ chế dịch tăng dần; cần đánh giá nó với cặp EN-VI thực tế. [4]

Bài tập: đưa một câu tiếng Anh vào từng cụm; ghi bản dịch thay đổi thế nào khi xuất hiện “not”, số thập phân hoặc mệnh đề cuối. So sánh dịch cụm với dịch câu đầy đủ.

Làm: nối NLLB với cụm EN ổn định; giới hạn lịch sử để không dịch lại cả buổi học. Thêm glossary 30 thuật ngữ môn học để hiển thị giải nghĩa; chỉ dùng thay thế thuật ngữ sau khi test, vì thay chuỗi mà có thể làm sai nghĩa.

Cổng M3: chạy 30 phút EN/VI, đạt chất lượng tối thiểu trang 9 và không dón queue. Sau đó A/B baseline với NLLW trên cùng audio. Chỉ chọn NLLW nếu giảm trễ mà không làm tăng lỗi nghiêm trọng.

07 / HỌC VÀ TRIỂN KHAI

Giai đoạn 4-6: từ demo đến dùng được

Ước lượng: 3-4 tuần, 20-32 giờ. Tổng dự kiến 8-10 tuần với khoảng 6-8 giờ/tuần.

Giai đoạn 4 / Web client và PWA

Học trước: JavaScript/TypeScript cơ bản, DOM, state machine, MediaDevices, AudioWorklet, HTTPS, manifest và service worker. Không bắt buộc React; UI đơn giản giúp học luồng dữ liệu dễ hơn.

Bài tập: mở mic sau thao tác chạm, hiện mức âm, Stop phải giải phóng track. Tắt mạng 10 giây rồi nối lại; kiểm tra iPhone Safari và chế độ Add to Home Screen.

Làm: EN/VI hai hàng theo đoạn; chữ lớn, tự cuộn có nút tạm dừng; trạng thái mic/mạng/server; EN-only khi dịch chậm. iPad vào cùng session với quyền chỉ xem. Chỉ cache app shell, không cache transcript mặc định.

Cổng M4: thu thật 30 phút trên iPhone và đọc trên iPad; từ chối mic, khóa màn hình, chuyển ứng dụng đều có hành vi được ghi nhận. PWA không đảm bảo thu nền: kế hoạch sử dụng là giữ trang ở foreground.

Giai đoạn 5 / Kết nối về dorm

Học trước: IP/DNS, TLS, reverse proxy, VPN, xác thực, Linux service. **Bài tập:** dùng mạng di động truy cập trang health qua tailnet, rồi ngắt laptop để xác minh UI báo lỗi.

Làm: Tailscale Serve HTTPS, service tự khởi động, log luân phiên, model cache local và cơ chế warm-up. Thử phiên 60 phút qua mạng ngoài dorm, sau đó thử Wi-Fi trường.

Cổng M5: URL HTTPS hợp lệ, WebSocket sống qua phiên thử; Stop dùng thu thật; người ngoài quyền không vào được phiên. Reboot máy rồi tự khôi phục dịch vụ.

Giai đoạn 6 / Release có bằng chứng

Học trước: regression test, p50/p95, CI, tag/release và tài liệu tái lập. **Bài tập:** cài từ checkout sạch và chạy test bằng README. **Làm:** 90 phút soak test, đo nhiệt/bộ nhớ/latency; quay demo bằng audio được phép. **Cổng M6:** đạt bảng trang 9, công bố v0.1.0 với hạn chế thực tế; nếu chưa đạt, giữ nhãn prototype và mở issue cụ thể.

08 / ĐÁNH GIÁ

Thước đo quyết định có mang vào lớp

Các ngưỡng bên dưới là mục tiêu v0.1 đề xuất. Hiệu chỉnh sau baseline; công khai mọi thay đổi ngưỡng.

Chỉ số	Cách đo	Mục tiêu ban đầu
EN partial latency	Từ cuối đoạn audio tương ứng đến lúc client render chữ đầu tiên.	$p50 \leq 2\text{ s}$; $p95 \leq 4\text{ s}$
VI final latency	Từ cuối cụm nguồn tương ứng đến lúc render bản dịch chốt.	$p50 \leq 4\text{ s}$; $p95 \leq 8\text{ s}$
Khả năng theo kịp	RTF = thời gian xử lý / thời lượng audio; thêm queue age theo thời gian.	RTF ASR < 0,7; queue không tăng dài hạn
ASR WER	(thay + xóa + thêm từ) / số từ tham chiếu; cố định chuẩn hóa.	$\leq 15\%$ sạch; $\leq 25\%$ audio phòng thử
Dịch đúng ý	Chấm mù 1-5 trên ≥ 100 cụm; kiểm phủ định, số và thuật ngữ.	$\geq 80\%$ cụm đạt 4/5; $\leq 5\%$ lỗi nghiêm trọng
Độ ổn định chữ	Số lần sửa mỗi partial; số final bị sửa không thông báo.	Final sửa im lặng = 0
Độ bền	90 phút end-to-end; theo dõi RAM/VRAM và disconnect.	0 crash/OOM; gap luôn được báo
Khôi phục mạng	Ngắt 10 s rồi nối; đo từ lúc mạng trở lại tới kết quả mới.	$\leq 10\text{ s}$; không nhân đôi final

RTF offline chỉ là bước sàng lọc; streaming có thể decode lặp. Luôn báo RTF live, queue age, số frame mất và latency cho phiên thực. Không đánh giá chất lượng chỉ trên các đoạn thành công; báo cả phần bị bỏ.

Để tránh lệch đồng hồ, dùng timeline sample audio và mapping về đồng hồ monotonic của client. Server ghi thời gian từng công đoạn; đo end-to-end tại client gửi mic. Với iPad, ước lượng offset bằng ping hoặc dùng video đối chiếu. Tách cold start, warm-up và latency ổn định.

09 / KIỂM THỬ

Bộ thử nhỏ, tái lập được

Dữ liệu và cách chấm

Tạo 30-45 phút audio được phép: giọng rõ, nói nhanh, xa mic, tiếng ồn, thuật ngữ STEM, số/đơn vị, phủ định và khoảng im lặng. Chia theo bản ghi thành tập chỉnh cấu hình và tập kiểm tra giữ kín; không dùng cùng đoạn để vừa tối ưu vừa báo điểm cuối.

Chép tay EN và dịch VI cho ít nhất 100 cụm, lưu ranh giới thời gian và nhãn lỗi. Nếu có thể, nhờ người song ngữ chấm mà không biết cấu hình; nếu tự chấm, ghi rõ hạn chế. Có thể thêm chrF qua SacreBLEU để so sánh tương đối, nhưng điểm tự động không thay được kiểm nghĩa.

Lớp test	Cá kiểm chứng cần có
Unit	Resample giữ đúng thời lượng; seq/gap; segment revision; flush cuối phiên; queue cap; ngôn ngữ nguồn/đích.
Integration	Audio fixture → ASR → MT → events; worker chậm không chặn heartbeat; reconnect không lặp kết quả.
Browser / thiết bị	Safari iPhone, Safari iPad và PWA; mic denied/revoked; rotate; khóa màn hình; app switch; đổi Wi-Fi/cellular.
Failure injection	Mất mạng 10/30 s; model lỗi/OOM; server restart; sai token; frame lớn/sai định dạng; subscriber không được gửi mic.
Soak / regression	90 phút ở nguồn điện và chế độ nhiệt thực; 10 câu lỗi khó cố định; kiểm bộ nhớ sau warm-up và lúc cuối.

Báo cáo mỗi lần benchmark

Ghi commit, model revision, backend, quantization, beam, chunk, VAD, số luồng CPU, driver/runtime, nguồn điện và điều kiện mạng. Chạy lặp 3 lần bộ ngắn sau warm-up; báo median và p95, không chỉ số tốt nhất. So sánh base/small và baseline/NLLW theo từng thay đổi một.

Cách tìm nguyên nhân: EN sai → xem mic/VAD/ASR. EN đúng, VI sai → xem segmentation/model dịch. Cả hai đúng nhưng chậm → xem queue, thời gian commit, CPU và mạng. Luôn sửa nút thắt đã đo.

10 / TRIỂN KHAI

Laptop ở dorm, client trong giảng đường

Thiết lập một lần

1. Boot Ubuntu; xác minh GPU. Dùng môi trường Python riêng và model cache trên phân vùng Linux. Dành khoảng 15-25 GB đĩa cho môi trường/model/cache ban đầu, rồi kiểm tra dung lượng thực. Giới hạn bộ nhớ ứng dụng để còn dư cho hệ điều hành; theo dõi swap và tránh nạp nhiều bản model.
2. Chạy backend tại loopback, ví dụ 127.0.0.1:8000. Cài Tailscale trên laptop, iPhone và iPad; dùng Serve làm HTTPS reverse proxy tới backend. Truy cập bằng hostname HTTPS của tailnet, kiểm chứng WSS; không bật Funnel cho dự án này. [8]
3. Hạn chế quyền tailnet cho thiết bị/người dùng của bạn. Thêm xác thực phiên ứng dụng, kiểm tra Origin WebSocket và phân vai sender/viewer. Tailscale bảo vệ đường truy cập; ứng dụng vẫn phải kiểm tra ai được vào session.
4. Dùng systemd chạy tài khoản thường: thư mục làm việc, biến môi trường, restart-on-failure và log rotation. /health kiểm tra tiến trình, /ready chỉ sẵn sàng sau khi model load và warm-up. Giới hạn worker suy luận là một để tránh nhân bản model trong VRAM.

Checklist trước mỗi buổi

Laptop cắm điện, thông gió, không suspend khi đang phục vụ; nếu đóng nắp phải thủ trước cấu hình nguồn. Dual boot sang Windows sẽ làm server Ubuntu dừng. Mở client, thử 30 giây audio, kiểm mic, mức âm, EN/VI, pin và kết nối từ mạng đang dùng. Giữ trang thu âm mở ở foreground.

Khi gặp sự cố

Wi-Fi trường có thể khiến đường truyền đi qua relay hoặc không kết nối được; đo trên mạng thật, đừng lấy tốc độ LAN làm chuẩn. Nếu lag, chuyển EN-only. Nếu server không tới được, báo offline và dừng upload; PWA cache chỉ giúp mở UI, không thay laptop suy luận. Phương án dự phòng là dùng laptop trực tiếp tại lớp nếu được phép.

Gói Tailscale Personal hiện có lựa chọn miễn phí; kiểm tra điều kiện tại lúc triển khai. Không coi dịch vụ bên ngoài là cam kết miễn phí vĩnh viễn. [9]

11 / QUYỀN RIÊNG TƯ

Giữ dữ liệu trong phạm vi cần thiết

Rủi ro	Biện pháp trong thiết kế
Thu âm khi chưa được phép	Xác nhận chính sách lớp trước khi bật mic. Có nút Stop rõ, chỉ báo đang thu và mặc định không lưu audio.
Lộ transcript hoặc audio	HTTPS/WSS qua tailnet; không public port. Transcript render như text, không chèn HTML từ model.
Thiết bị mất hoặc phiên bị dừng nhầm	Thu hồi thiết bị trong tailnet; session có quyền riêng, token ngắn hạn; không để token trong URL/log.
Log vô tình chứa lời giảng	Log kỹ thuật chỉ ghi ID, timing, lỗi và tài nguyên. Chế độ debug nội dung phải opt-in, tự hết hạn.
Tồn dữ liệu ngoài ý muốn	Buffer audio chỉ trong RAM và có giới hạn. Không bật transcript cache trong service worker; có nút xóa phiên.
Lạm dụng endpoint	Kiểm schema, kích thước frame, tốc độ gửi và session limit. Chỉ một sender; từ chối người chưa xác thực.

Chính sách lưu trữ đề xuất

Mặc định xóa trạng thái phiên khi kết thúc. Nếu bật lưu transcript, hiển thị vị trí, mục đích và hạn giữ, ví dụ 7 ngày; có tác vụ dọn và kiểm tra việc xóa. Xuất TXT/SRT chỉ theo thao tác người dùng. Giữ benchmark trong thư mục riêng, dùng dữ liệu đã cho phép.

Không commit bản ghi lớp, transcript thật, cookie, token, .env hoặc model weights. Nếu có backup, áp dụng thời hạn lưu/xóa tương ứng; xóa file không tự xóa bản sao backup. Lấn tải model cần mạng và có thể tạo metadata ở nhà cung cấp; sau đó kiểm thử inference bằng cache local.

Giấy phép và cách công bố

NLLB 600M có trọng số CC-BY-NC-4.0 và model card nêu các giới hạn sử dụng/chất lượng; giữ attribution và điều kiện tương ứng. [3] Kiểm giấy phép riêng của từng thư viện và dữ liệu. Chọn giấy phép cho phần code tự viết, thêm THIRD_PARTY_NOTICES; không tuyên bố toàn bộ stack được phép thương mại chỉ vì repo có MIT.

12 / GITHUB

Biến mini project thành hồ sơ kỹ thuật

Tên repo gợi ý: **lecturebridge**. Tạo repo khi bắt đầu triển khai; tài liệu này chưa tạo repo hoặc thay đổi tài khoản GitHub của bạn.

Đường dẫn đề xuất	Vai trò
apps/server/	FastAPI, auth, session, WebSocket; cấu hình runtime.
apps/web/	Mic, UI song ngữ, viewer, manifest, service worker.
src/lecturebridge/	audio/, asr/, translation/, pipeline/, protocol/; adapter WLK.
tests/	unit/, integration/, e2e/; fixture tự tạo dung lượng nhỏ.
benchmarks/	Script replay, manifest dữ liệu, cấu hình và kết quả đã làm sạch.
docs/	architecture.md, setup-ubuntu.md, deployment.md, evaluation.md, adr/.
.github/	workflows/ci.yml; issue template và pull request template.
Gốc repo	README, LICENSE, THIRD_PARTY_NOTICES, SECURITY, CONTRIBUTING, .env.example, .gitignore, lockfiles.

Quy trình cho một tính năng

Issue có vấn đề và tiêu chí xong → nhánh feat/ws-reconnect → commit nhỏ → pull request có ảnh/demo và kết quả test → tự review diff → CI đạt → squash merge vào main. Không cần thêm nhiều nhánh dài hạn cho dự án một người.

CI mỗi PR: lint/type check, unit test, build web và integration dùng fake model/fixture nhỏ. Test GPU thật chạy thử công trên laptop, đính kèm báo cáo đã xóa dữ liệu riêng tư; không để PR không tin cậy chạy tự động trên laptop cá nhân.

Các mốc release

v0.0.1: offline baseline và số liệu. **v0.0.2:** EN streaming. **v0.0.3:** EN/VI web. **v0.1.0:** chạy từ iPhone/iPad qua tailnet, đạt test 90 phút. Mỗi tag ghi cấu hình model/runtime và hạn chế; lưu release trước để rollback.

README chuyên nghiệp: vấn đề → demo được phép → sơ đồ → yêu cầu máy → cài và chạy → benchmark tái lập → quyền riêng tư → giới hạn → roadmap. Ưu tiên bằng chứng hoạt động hơn huy hiệu trang trí.

13 / NGUỒN & BƯỚC ĐẦU

Tài liệu để học đúng phần cần dùng

Nguồn sơ cấp đã đối chiếu ngày 12.09.2026. API và tùy chọn có thể thay đổi; ghim release/commit khi triển khai.

[1] **WhisperLiveKit**

Đọc README và cấu hình backend; chạy demo trước khi tùy biến.

[2] **Faster-Whisper**

Đọc GPU requirements, quantization, VAD và cách đo benchmark.

[3] **NLLB-200 distilled 600M - model card**

Đọc intended use, giới hạn model và giấy phép trọng số.

[4] **NoLanguageLeftWaiting (NLLW)**

Đọc stable prefix / LocalAgreement và giao diện dịch online.

[5] **Silero VAD**

Học phát hiện lời nói và thực nghiệm với im lặng/tiếng ồn.

[6] **FastAPI - WebSockets**

Làm bài echo, disconnect và kiểm tra quyền kết nối.

[7] **MDN - getUserMedia**

Học quyền mic, secure context và xử lý lỗi thu âm.

[8] **Tailscale Serve**

Thiết lập HTTPS nội bộ và kiểm tra access control.

[9] **Tailscale - Pricing**

Xác nhận điều kiện gói Personal tại thời điểm triển khai.

Buổi học đầu tiên, 90 phút: tạo thư mục dự án và nhật ký học; kiểm GPU trên Ubuntu; tự thu một WAV ngắn; ghi sample rate/thời lượng; chép EN và VI tham chiếu. Kết thúc bằng một commit nhỏ và danh sách điều chưa hiểu.

Tài liệu này đề xuất thiết kế và bài tập, không xác nhận phần mềm đã được cài hoặc hiệu năng đã được kiểm chứng trên Lenovo của bạn. Các thiết lập buffer, ngưỡng kiểm thử và lịch học là lựa chọn kỹ thuật để bắt đầu, cần điều chỉnh bằng kết quả thực nghiệm.